

AGENT

どこに何があるか、から始める

Claude Code の技術キーワードと仕組みを知って、自分の環境を仕込む

コードを書かずに作った、二つの実物

一行も書かず、日本語で頼んだだけ

実物 1

音声入力ツール

しゃべった言葉を文字にする。音声は外に出さず、手元のパソコンの中だけで処理が終わる。

実物 2

イベントログ+タスク管理

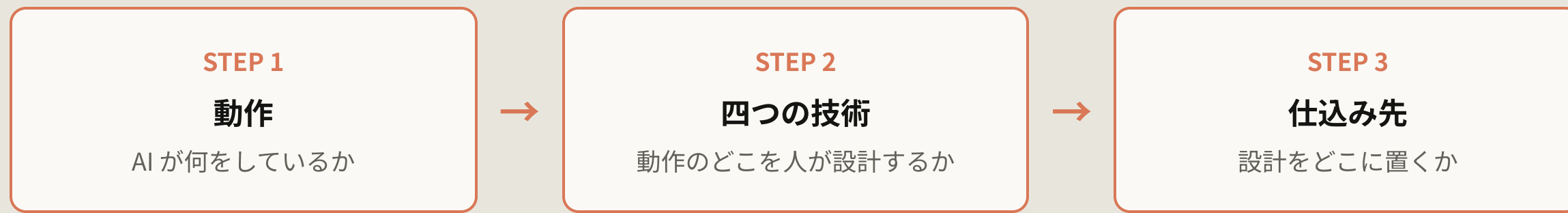
現場の出来事を記録し、
いま抱えている問題と着手順を出す。
実務で試している最中。

けれど、作る間ずっと同じことを打ち込んでいた

回答は日本語で。表はこの形式で。作業用のフォルダは触らないで。一日に何度もだった。今日は、繰り返しをやめる方法を扱う。

今日の到達点

今日の到達点は、どこに何があるか分かること



用語は平易な言い換えとセットで、初めて出るときに一度だけ説明する。用語だけだと現場で説明できず、日本語だけだと後で調べられないため。最後の24分は手を動かし、自分の環境に最初の一行を仕込む。

基本動作と、止まる場所

頼む→計画→実行→差分を見て承認



つまり、止まる場所は設定で選べる。最初は Manual を勧める。聞かれる面倒が、何が起きているかを知る勉強になるため。
自社の既定モードは登壇者が確認して伝える。

差分を見る

差分は、新旧と狙いの二つの見比べ

機械が出せる差分

- 変更前後の見比べ
- 消えた行と増えた行が赤と緑で出る

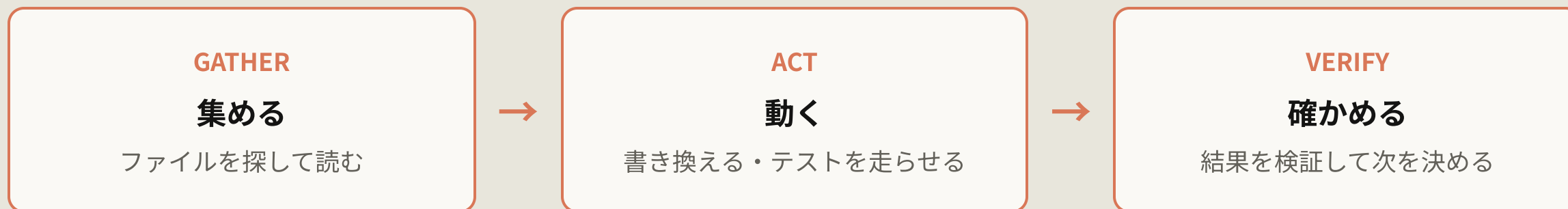
機械が出せない差分

- やってほしかったことと、出てきたもののギャップ
- 人が見るしかない部分

つまり、見る場所を先に二つだけ決めておく。① 触ってほしくないものに触っていないか ② 頼んだとおりの形になっているか。決めていないと毎回全部を読むことになり、そのうち読まなくなる。違っていたら承認せず、違う一か所だけを言い直す。

エージェントループ

集める→動く→確かめるを繰り返す



🔄 一周ごとに、前の結果を見て次を決める

頼んでから結果が出るまでにツールを何十回も呼ぶ。だから途中でずれると、ずれたまま進む。
確かめるところまで含めて一周になる。

つまり、人が入る理由は基準にある。AI は自分の出力を確認できるが、確認の基準は渡されたもののの中にしかない。
自社でやってはいけないこと、という基準は、渡さない限り存在しない。

つまり 一人が見る場所を決める

人が見る場所は、自分で決められる

01

その場で切り替える

権限モードを切り替える

02

設定に常設する

settings.json の権限で宣言する

03

機械的に止める

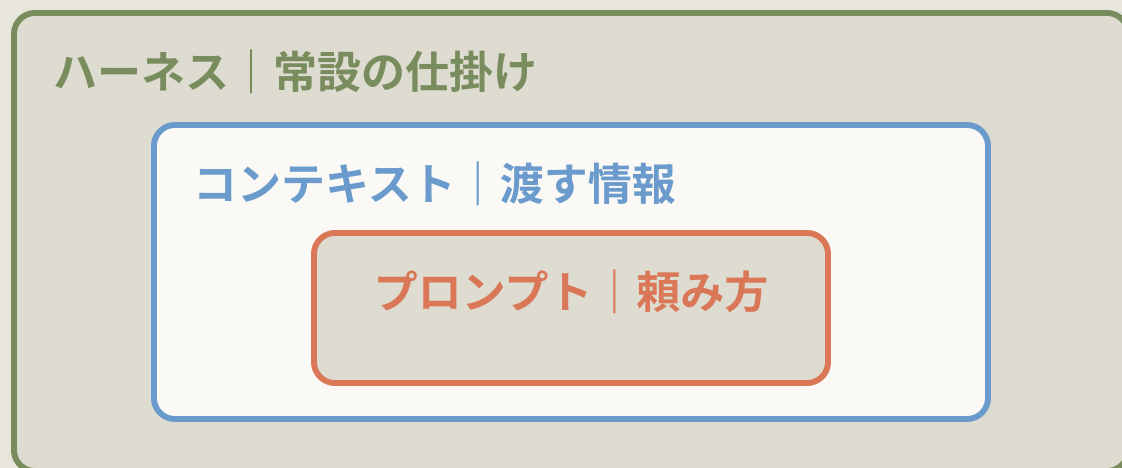
hooks で例外なく止める

手軽なものほどその場限り、強いものほど常設。今日は手軽なほうから順に見ていく。

四つの技術

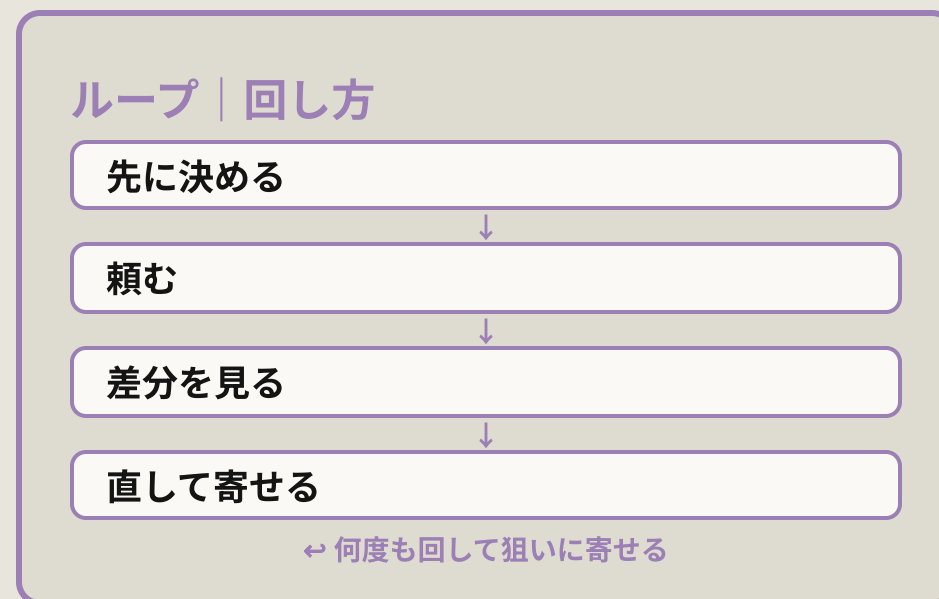
頼み方・渡す情報・常設の仕掛け・回し方

図1 | 四つの技術は入れ子になっている



外側にあるものほど、毎回きいてくる

図2 | 四つを回すのがループ



つまり、毎回言っていることは外側へ移していく。四つは「～エンジニアリング」と呼ばれることもあるが、この分野で使われている呼び方であり、公式が四分類を定義しているわけではない。

四段階のどこを設計するか

四つの技術は、四段階のどこを設計するかの話

動作の四段階	設計する技術（人間側）	具体で言うと
① 頼む	プロンプト 頼み方	依頼文の書き方そのもの
② 計画・③ 実行	コンテキスト 渡す情報	AI が読めるファイルと決まりごと
①～④ の土台	ハーネス 常設の仕掛け	毎回打たずに済むよう環境へ置く
④ 確認 → もう一周	ループ 回し方	違えば言い直して、もう一周

つまり、四段階は AI が何をしているか、四つの技術は、そのどこを人が設計するか。迷子になったら、四段階は AI 側、四つは人間側と思い出す。

依頼文の構造

依頼文は役割・背景・例・形の四点

× 「点検データをいい感じに整理して」 — 書いていない部分は推測で埋まる

役割

あなたは製造現場の品質管理を手伝う担当です

背景

月次の報告会で使う資料の下ごしらえをしたい

指示

点検記録を設備ごと・月ごとに、異常が何件あったか数えて

形と例

表の形で。例：設備A ／ 7月 ／ 3件

つまり、上から下へ、だんだん具体になる順に置く。役割が以降の読み方を決め、背景が推測の基準になり、そのうえで指示、最後に形と例。

フューショット

お手本は三～五個、囲って渡す

<example>

設備A ／ 7月 ／ 3件

設備B ／ 7月 ／ 0件

</example> ← 囲むと、例なのか依頼なのかを区別できる

個数

三～五個

一つだと特徴に引きずられ、
多すぎても頭打ち

渡し方

タグで囲う

指示と例の境目がはっきりする

効く理由

体裁が伝わる

表の形や言葉づかいの硬さは、
文章で書くより速い

役割設定と構造化

役割は一文、部品は見出しで分ける

役割設定（＝立場を一文で）

- 「あなたは品質管理を手伝う担当です」
- 一文でも、返ってくる説明の粒度が変わる
- 人に頼むとき無意識にやっていることを、明示するだけ

構造化（＝見出しで分ける）

- 背景 ／ やってほしいこと ／ 出してほしい形
- 見出し三つでも、だらだら書くより安定する
- 公式は XML タグでの区切りを勧めている

つまり、どちらも AI に推測させる余地を減らす手当て。身構えて覚える型ではなく、部品が分かれていると伝わればよい。

つまり — 曖昧さを減らす

曖昧さを減らすほど、出力は安定する

依頼文の四点	消える曖昧さ
役割	どういう前提で答えるかの曖昧さ
背景・目的	何を優先して判断するかの曖昧さ
例	どんな体裁で出すかの曖昧さ
構造	どこからどこまでが何かの曖昧さ

四点はどれも、依頼する側がすでに持っている情報。AI が知らないだけ。ただし毎回書くのは続かない。だから次は、渡す情報そのものの設計へ。頼み方の技術は、四段階のうち段階①の設計にあたる。

コンテキストウィンドウの中身

器の中身は指示・ファイル・応答の全部

指示

自分が打った指示

依頼文・訂正・やり取りの全部

ファイル

読み込んだファイルの中身

@ で渡したもの、AI が自分で開いたもの

応答

返ってきた応答

画面に出た文章も器に残る

見えない分

画面に出ていないもの

「設備」で検索した15件の中身まで入る

つまり、器は体感よりずっと早く埋まる。中身は /context で確認でき、どの CLAUDE.md が読み込まれたかも分かる。

あふれるとどうなるか

あふれる前に、古い所は自動で畳まれる

起きること

古い所が要約されて畳まれる

- エラーにはならないので気づかない
- 最初のほうに伝えた条件ほど薄くなる
- 「元データには触らないで」
が後半で効かなくなる

対策は二つ

区切る／外に出す

- 一段落したら会話を切り、
大事な条件だけ渡し直す
- 大きな読み取りはサブエージェントに任せる
- 任せた中身は、自分側の器に入らない

つまり、一度だけ口で伝えた条件は消える前提で扱う。消えてほしくないものは、次の章で扱うファイルに書く。

@参照で名指しする

@でファイルを名指しして渡す

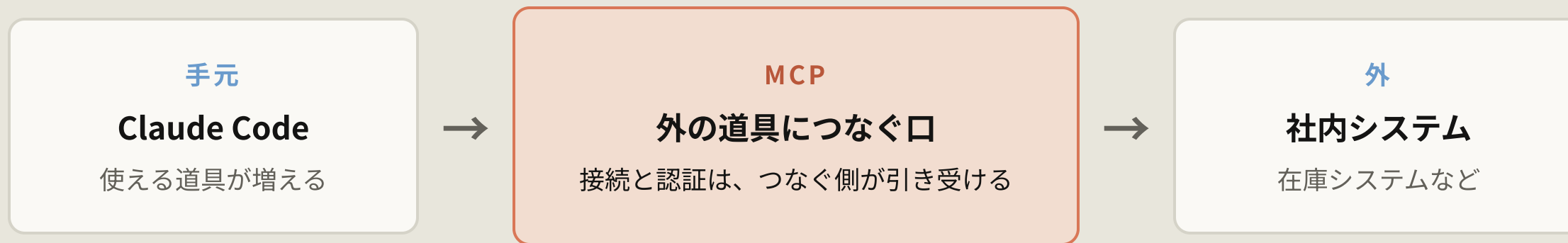
@data/inspection_2607.csv を見て、設備ごとの異常件数を数えて

- 名指しなので、似た名前のファイルがあっても読み間違えない
- 大きいファイルは、必要な部分だけを切り出して渡すほうが効く
- 社外に出せないものは器に入れない。会社の規定が優先

つまり、渡さない判断も設計のうち。今日のワークは自分の手元のファイルだけで足りる。

MCP — 外の道具につなぐ口

MCP は外の道具につなぐ口



つまり、つなぎ方を覚える必要はない。誰かが口を用意すれば、Claude 側からは道具が増える。増えた道具も、使う前に明示的な許可が要る。勝手に外を触りに行くことはない。

つまり — 器を設計する

入れるものと入れないものを選ぶ

器に入れる

- 今回の指示
- 判断に要る決まりごと
- 必要な部分だけ切り出したファイル

器に入れない

- 関係のない資料
- 古いやり取り
- 社外に出せないもの

渡す量と精度は比例しない。片っ端から渡したら、肝心の指示が資料に埋もれて的外れになった。渡す情報の技術は、四段階のうち段階②③の材料の設計にあたる。

.claude の中に何があるか

仕込みの置き場は .claude の中にある

```
~/claude/ ← ホーム側 | 自分の全プロジェクトに効く
├── CLAUDE.md 毎回読まれる指示書き（自分の好み）
└── settings.json 権限と設定
```

```
your-project/
├── .claude/ ← プロジェクト側 | git でチームに共有できる
│   ├── CLAUDE.md このプロジェクトの決まりごと
│   ├── rules/ パスを限った規則
│   ├── skills/ 手順のパッケージ
│   └── settings.json 権限と止めどころ
```

つまり、置き場所は二系統。自分の好みはホーム側へ、チームで共有する決まりごとはプロジェクト側へ。区別を誤ると、個人的な好みが他の人の環境にも効いてしまう。全部を最初に作る必要はない。

CLAUDE.md に何を書くか

毎回説明し直すことを書いておく

書く

- 手順 — テストは `npm test` で走らせる
- 置き場所 — API のコードは `src/api` に置く
- 決まりごと — コミットメッセージは日本語で

書かない（別の場所へ）

- 何段階もある手順 → `skills` へ
- 一部のファイルにだけ関わる規則 → `rules` へ
- 分量そのもの — 目安は1ファイル200行以内

書きどきは、同じ説明を二度目に打ち込んだとき。書き方は検証できる具体性で。「きれいに整形する」ではなく「インデントは半角2つ」、「テストして」ではなく「コミット前に `npm test` を走らせる」。
曖昧な指示は曖昧にしか守られない。

CLAUDE.md はいつ読まれるか

四つの置き場所が、順に連結される

組織

管理者が置く層

個人の設定では外せない

ユーザー

ホームの CLAUDE.md

自分の全プロジェクトに効く

プロジェクト

git で共有される層

チーム全員に効く

ローカル

CLAUDE.local.md

自分だけ。git には入れない

上書きではなく連結される。組織 → ユーザー → プロジェクト → ローカルの順に並び、毎セッション読まれる。
ただし強制設定ではなく文脈として扱われる。書いたのに守られないことは起こり得る。必ず止めたいならフックを使う。

近い場所ほど後に読まれる

近い場所ほど後に読まれ、強く効く

①

上へたどる

作業フォルダから上へ集める



②

順に並ぶ

ルートに近いほうから並ぶ



③

後ほど強い

近いものほど後に読まれる

サブフォルダ

読まれるのは触ったとき

起動時には読まれない。

そのフォルダのファイルを AI が読んだとき、
初めて読み込まれる（遅延ロード）

圧縮のあと

直下だけが戻る

圧縮されて戻るのはプロジェクト直下だけ。
サブフォルダやパス限定の規則は戻らない

つまり、大事なことほどプロジェクト直下を書く。口頭で伝えた指示は、圧縮されると消える。

rules と skills の使い分け

規則は常設、手順はスキルに切り出す

置き場所	いつ読まれるか	向いているもの
CLAUDE.md	毎セッション、必ず	短い決まりごと・置き場所・毎回の手順
.claude/rules/	パスの条件に合うファイルを触ったとき	特定の領域だけの規約
.claude/skills/	必要になったとき、名前を呼んだとき	三段階以上の手順のパッケージ

つまり、いつ読まれてほしいかで決める。パスを限れば、常に全部を読ませなくて済むので器の節約にもなる。

CLAUDE.md の一節が「事実」ではなく「手順」に育ったら、スキルへ切り出す合図。

settings.json — 権限と止めどころ

触れる範囲と止めどころを設定で決める

CLAUDE.md = お願い

- 文脈として伝わるだけ
- 守られないことは起こり得る
- 書きすぎると、かえって守られにくい

settings.json = 宣言

- ツールを使う前に許可ルールと突き合わせる
- どのツールをどこまで使ってよいかを決める
- 読み取りは通す、消す操作は必ず確認

つまり、絶対に触られたくない場所があるなら、お願いではなく設定に書く。使い分けだけ覚えて帰ればよい。
設定ファイルの書き方は持ち帰り資料に載せる。

hooks の三階層（イベント → 対象 → 処理）

決まった場面で、決めた処理を自動で走らせる

```
{ "PreToolUse": [      ← ① イベント | どの場面で
  { "matcher": "Bash", ← ② 対象 | 何に反応するか
    "hooks": [ … ] } ] } ← ③ 処理 | 何を走らせるか
```

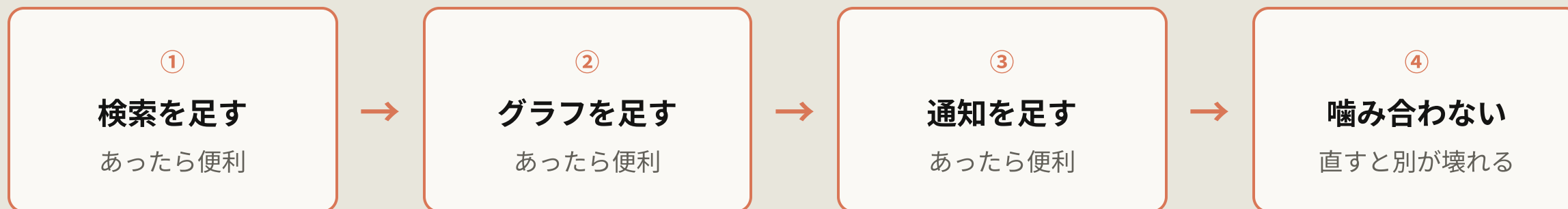
実例 | 危ない削除コマンドの手前で止める

イベントはツールを使う前、対象はコマンド実行、処理は内容を検査して危なければ止める。
人が注意していようがいまいが、機械的に止まる。

つまり、規律で守るを、仕組みで守るに変える道具。書き方まで今日覚える必要はない。

発散の罠

足すほど目的がぼやけて使えなくなる



原因は一つ — 目的が決まっていない

目的が決まっていなければ、何が正解かも、何ができたら終わりかも決まらない。終わりが決まっていない作業は終わらない。そして作るのが速い分だけ、発散も速い。手で作っていた頃なら面倒で足さなかったものが、簡単に足せてしまう。

先に決める三つ（SDD）

作るもの・完成条件・止めどころを決める

01

何を作るか

一文で言い切れる大きさに

02

何ができたら完成か

見て丸かバツか付けられる形に

03

どこから自分が見るか

止まる場所を先に決める

実際に書くと三行

点検記録から設備ごと月ごとの異常件数を表にする / 設備ごとの件数が月別に並んだ表が出たら完成 /
元データには触らない

先に仕様を決めて作る型は SDD（スペックドリブン開発）と呼ばれる。書くのに一分もかからない。

Plan モードで受ける

Plan モードは調べて計画するだけ



効くのは、完成条件を決めてあるから

表が出たら完成、と決めておくと、計画にグラフが入った瞬間に落とせる。決めていないと「あってもいいか」で通してしまい、膨らんだ末に発散する。実装まで進んでから気づくと戻すのが面倒。計画の段階なら一言で済む。

つまり一回して寄せる

先に決めるのは、直す物差しを持つため

× 一発で当てるため

- 最初に全部決めて一直線に完成させる、ではない
- 途中で違ったと分かることは必ずある
- 早めに出して、見て、直すほうがうまくいく

○ 直す物差しを持つため

- 何を作るか・何ができたら完成かが決まれば
- 出てきたものが狙いに近いか遠いか判断できる
- 物差しがないと、直すつもりが迷走になる

先に決めるのは三行で足りる。分厚くすると、今度は仕様書を書くこと自体が目的になる。回し方の技術は、四段階のうち段階④から次の一周へ戻るところの設計にあたる。

ワークの進め方

24分で最初の一行を仕込む

01

何を書くか決める

同じ説明を二度以上していること

02

CLAUDE.md に書く

今日触っているプロジェクトの直下

03

/context で確認

自分の CLAUDE.md が出ていれば成功

例を三つ。言葉づかい「回答は日本語で書く」／触ってほしくない場所「data 中のファイルは変更しない」／
毎回の手順「変更したら必ずテストを走らせる」。書き方は具体的に。思いつかない方、詰まった方は手を挙げてください。

各自ワーク

自分の .claude に一つ入れる

24分 — 自分の .claude に、最初の一行を入れる

- 一行でよい。動くツールを作る時間ではない
- 詰まったら手を挙げてください。巡回します
- 余裕のある方は settings.json の権限か、skills への切り出しへ

持ち帰る地図

持ち帰るのは、地図と最初の一行

動作の四段階	設計する技術
① 頼む	プロンプト 頼み方
② 計画・③ 実行	コンテキスト 渡す情報
①～④ の土台	ハーネス 常設の仕掛け
④ 確認 → もう一周	ループ 回し方

次の一手

短い決まりごとが増えたら rules へ。三段階以上の手順になったら skills へ。
絶対に止めたいものが出てきたら settings.json の権限か hooks へ。